It Takes Two to Tangram: Multi-Agent Object Centric Reinforcement Learning for Assembly

Anonymous Authors

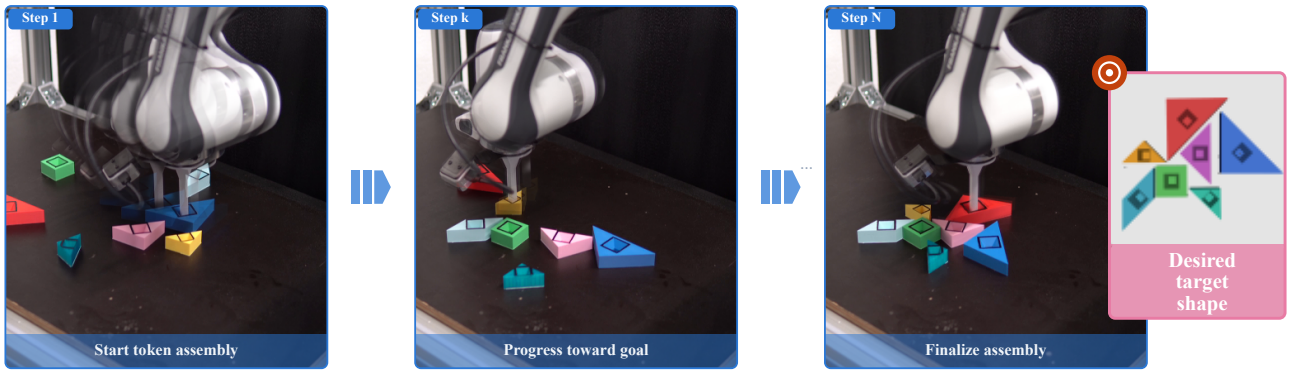


Fig. 1: GOMA solving the tangram puzzle on our TangramBench. Given the target assembly (right), we focus on the complex task of coordinating feasible object motions constrained to the support plane. GOMA predicts object motions that avoid deadlocks and collisions under varying clearance to achieve the final assembly.

***Abstract*— Geometric assembly requires fine spatiotemporal coordination to place parts without causing collisions or deadlocks under tight clearances. To address this, we present an object-centric reinforcement learning framework that treats each part as an agent and learns a shared policy for continuous assembly control. A graph neural network represents the pieces, their target poses, and their spatial relationships, and predicts translational and angular velocities for each component. Per-agent proximal policy optimization with a centralized graph critic directly learns motion and temporal coordination within a single feedback policy. We study this formulation through planar Tangram assembly, where pieces must slide and rotate into a given configuration while remaining on the assembly surface. Procedural generation of target configurations and randomized initial arrangements support training and evaluation across diverse assembly problems. Our experiments isolate the roles of relational structure and multi-agent optimization, and compare learned control with sampling-based planning. We demonstrate concurrent assembly in simulation and transfer the policy to sequential execution on a real robot, as our object-centric formulation decouples learned assembly coordination from low-level physical control.**

I. Introduction

From arranging or assembling furniture in a home to fitting components together on a production line, many tasks require moving multiple objects into precise spatial relationships, often within cluttered or confined spaces [1], [2]. In such cases, even when every object's destination is known, its motion cannot be considered in isolation. To illustrate the problem, consider sliding puzzle pieces across a table to form a given shape. As one piece approaches its target, it may narrow or block the path of another. Once the desired configuration is known, the challenge becomes coordinating the pieces' motions toward their targets while accounting for the space they need along the way.

One approach to the coordination problem is to jointly plan which objects to move and how to move them. Task and motion planning (TAMP) methods couple these decisions by searching for action sequences whose associated motions satisfy geometric and robot constraints [3]. In robot assembly, this involves selecting part sequences and assembly configurations together with feasible manipulation motions [4]. In nonprehensile rearrangement, object-centric approaches guide planning through actions directed at individual objects [5], or compute desired object motions that are realized through closed-loop pushing [6]. This abstraction organizes manipulation around changes to the objects whose configurations define the task. Learning-based approaches also address multi-object rearrangement through hierarchical policies [7], while assembly methods exploit structured representations of parts and their relationships to select placements [8] or generate object motions [9]. These approaches establish objects and their relationships as useful units for reasoning about manipulation.

Multi-agent pathfinding offers another perspective on coordination, studying how agents can reach their respective goals while avoiding conflicts during concurrent movement [10]. Coordinating these movements requires reasoning about interactions between agents and the space they share. Graph representations provide a way to encode these relationships, supporting learned policies for decentralized path planning [11], multi-agent navigation and collision

Claude (Anthropic) was used for grammar polishing and for coding assistance in the experiments (Section IV) and in generating the splash images in Fig. 1. All outputs were reviewed and verified by the authors.

avoidance [12], and coordinated motion, task allocation, and scheduling in multi-robot reaching [13]. This perspective naturally extends to object-centric assembly and rearrangement, where individual objects can be represented as agents acting toward a shared target configuration.

Thus, we introduce Graph networks for Object-centric Multi-agent Assembly (GOMA), a graph-based multi-agent approach to geometric assembly. GOMA represents each part to be assembled as a cooperative agent and uses a shared policy trained with Multi-Agent Proximal Policy Optimization (MAPPO) [14] to predict continuous object motions from a relational representation of the assembly state. A graph neural network (GNN) encodes interactions between parts, combining an object-centric representation with multi-agent learning toward a shared assembly objective. Our central question is whether an object-centric multi-agent formulation provides an effective basis for learning coordinated assembly. We further investigate which aspects of the representation and learning formulation matter for coordination, including how information about other parts informs each agent's actions.

To study this problem, we also introduce TangramBench, a procedural benchmark suite for planar geometric assembly based on the game of Tangram. The classical Tangram puzzle consists of seven geometric pieces (see Fig. 1) that must be arranged to form a target silhouette. Despite its small set of simple parts, tightly packed configurations create strong geometric dependencies between their motions, making Tangram a compact setting for studying coordinated assembly. Tangram has previously served as a robot testbed for collaborative pick-and-place [2] and sequential assembly from silhouette prompts [15], while recent benchmarks study geometric reasoning over valid piece configurations [16], [17]. In contrast, TangramBench provides the target pose of each piece and constrains its motion to the support plane, isolating the problem of coordinating feasible object motions toward a known assembly. We procedurally generate three levels of geometric difficulty, from loosely packed configurations to compact assemblies in which interactions between pieces become increasingly important. The benchmark supports large-scale training and evaluation and includes simulation-ready models together with physical pieces and a compatible manipulation tool for real-robot experiments.

In summary, our contributions are:

- An object-level multi-agent formulation of geometric assembly that learns simultaneous part motions and temporal coordination through a shared continuous-control policy.
- An empirical investigation of relational state representations and per-agent policy optimization, with comparisons against joint-action learning and sampling-based planning.
- TangramBench, a procedural benchmark suite for geometric assembly, with physical pieces and evaluations of concurrent simulation control and sequential robot execution.

II. Related Work

A. Assembly and Motion Planning

Task and motion planning couples discrete task decisions with continuous geometric constraints [3]. Assembly planners exploit disassembly to recover feasible sequences and motions, accounting for part geometry, stability, and graspability [4], [18]. Fabrica extends this approach with learned controllers for contact-rich execution [19], while [20] coordinate concurrent construction through optimization and space-time search over smaller task subproblems. These approaches explicitly search for feasible assembly operations and their execution. Additionally, sampling-based methods such as RRT-Connect [21] construct paths through collision-free configurations. Learned state and action proposals can guide physics-based rearrangement planning [22], and learned tree-search policies can select objects and pushing primitives [7]. Hierarchical policy blending combines reactive control with planning over the weights of expert policies [23]. Our approach instead learns a feedback policy that coordinates part velocities from the assembly state without an online search over object trajectories, manipulation primitives, or controller weights.

B. Learning for Assembly and Rearrangement

Several methods have applied learning algorithms to assembly tasks. Learn2Assemble combines structured reinforcement learning and search with geometric and robot feasibility [8]. Other methods learn feasible assembly sequences [24] or alternative plans under execution disruptions [25]. Geometric assembly methods also infer the desired part arrangement. Dynamic graph learning predicts part poses from geometric input [26]. Particularly for geometric assembly, MRChaos learns visual Tangram assembly through sequential pick-and-place actions in a pre-defined part order [15]. Blocks Assemble! [9] uses a graph policy to predict block velocities and select blocks for two virtual grippers, with large-scale proximal policy optimization (PPO) training across diverse blueprints. Kataoka et al. [27] transfer joint-space reinforcement learning to bimanual magnetic block assembly. Conversely, our approach uses object relations to coordinate continuous part velocities, coupling movement and temporal ordering throughout execution rather than only when selecting operations. Here we take assigned target poses as input and learn the planar motions. During execution, pieces remain on the assembly surface so that their shapes and poses constrain one another. Overall, our approach, differs from previous geometric assembly works as we learn the object-level velocities with a multi-agent learning formulation, assigning a policy output and optimization objective to each piece while retaining a shared graph representation that supports both concurrent assembly in simulation and sequential execution with a real robot.

C. Graph Policies and Multi-Agent Coordination

Coordinating multiple agents requires accounting for how each agent's motion affects the others. Multi-agent pathfinding studies this problem through interacting routes and

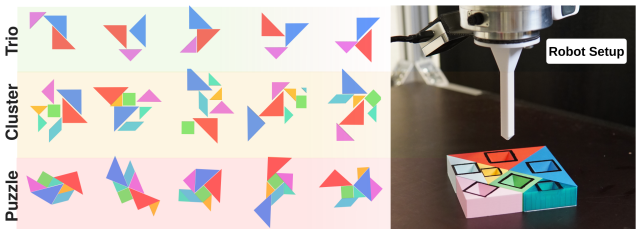


Fig. 2: TangramBench provides a graded benchmark for planar geometric assembly, ranging from the three-piece TangramBench-Trio to seven-piece TangramBench-Cluster and tightly constrained TangramBench-Puzzle configurations. We provide simulation-ready models, together with a robot-compatible physical counterpart consisting of 3D-printed pieces and an insertion tool for planar manipulation.

temporal collision constraints [10]. In learned policies, graph networks provide a natural way to represent these interactions through shared relational computations [28]. InforMARL [12], for example, combines graph-based information aggregation with MAPPO [14] to learn navigation policies with centralized value estimation. RoboBallet [13] applies graph-based coordination to multiple robot arms, jointly addressing task allocation, scheduling, and motion. We adopt a similar perspective for assembly, treating the pieces themselves as cooperative agents. Here, our approach moves the coordination problem from robot arms to assembly pieces, using a shared graph policy to learn coupled motions whose feasibility depends on the shapes and poses of the other parts as they form the final structure.

D. Robot Assembly Benchmarks

Assembly benchmarks span tasks ranging from determining valid part configurations to executing the manipulation required to realize them. Tangram provides a particularly compact instance of this problem and has been used across several settings. Kirschner et al. [2] use a five-piece Tangram for collaborative robot assembly, where pieces are grasped and placed at taught target locations, while Zhao et al. [15] learn sequential pick-and-place assembly of the seven Tangram pieces from a target silhouette. Outside physical manipulation, generated Tangram datasets have been used to learn mappings from silhouettes to complete solutions [29], including TANGAN with thousands of randomly generated puzzles [29]. These works primarily evaluate *which configuration* solves a puzzle or the accuracy with which pieces can be placed. In contrast, TangramBench takes the desired configuration as given and evaluates the coordinated planar manipulation required to reach it from an initial arrangement, making interactions and conflicts between object motions part of the benchmark itself.

III. TangramBench

TangramBench is a procedural benchmark for planar geometric assembly built around the seven pieces of the classical Tangram puzzle. As illustrated in Fig. 2, the set consists of two large triangles, one medium triangle, two small triangles, a square, and a parallelogram. Although each individual piece has only three planar degrees of freedom (position and yaw), assembling several pieces into a compact configuration gives rise to a coupled, constrained motion-planning problem. As target slots become occupied, the collision-free space available to the remaining pieces changes and may contain increasingly narrow passages. A motion that is locally valid for one piece can therefore obstruct the route of another, making successful assembly depend not only on reaching the individual targets, but also on coordinating how the pieces approach them.

For TangramBench we therefore consider a set of N rigid planar pieces, with the configuration of piece i given by

$$q_i = (p_i, \vartheta_i) \in \mathrm{SE}(2), \qquad p_i = (x_i, y_i) \in \mathbb{R}^2, \qquad (1)$$

and denote the joint configuration of all pieces by $q = (q_1, \ldots, q_N)$. Each task instance therefore specifies an initial configuration q^0 and an assigned target configuration $q^\star = (q_1^\star, \ldots, q_N^\star)$. The objective here is to drive all pieces from q^0 to their corresponding targets while keeping them within the support plane and avoiding inter-piece penetration throughout the motion.

Because the piece-to-target assignment is provided, TangramBench deliberately removes assignment and shape-matching ambiguities and isolates the coordination problem. That is, TangramBench central question is:

How should the pieces move to their known targets without preventing one another from reaching them?

We show in Fig. 2 an overview of our benchmark suite. For each instance, target poses $q_i^\star = (p_i^\star, \vartheta_i^\star)$ are placed on a 70×36cm support plane. Initial poses are sampled within a 60×34cm region, with piece centers separated by at least 18cm. Pieces may translate and rotate freely on the plane. We consider a piece successfully placed when its center is within 1cm of the target position and with an orientation error below 0.1rad. To expose different degrees of geometric coupling, TangramBench contains three levels of increasing difficulty. (i) **TangramBench-Trio** contains only three pieces—the two large triangles and the medium triangle—and provides a reduced setting in which coordination can be studied without the constraints induced by the full puzzle. (ii) **TangramBench-Cluster** uses all seven pieces and generates compact target configurations procedurally. Pieces are first scattered and then attracted toward a common center until contact prevents further motion, with configurations containing interpenetrations rejected. This process produces irregular but relatively open assemblies, for which most target slots remain accessible from outside the configuration. Finally, (iii) **TangramBench-Puzzle** contains tightly packed Tangram configurations constructed by joining pieces edge-to-edge at compatible orientations, making them closer to classical Tangram figures. These layouts create enclosed target slots and narrow access corridors, such that placing one piece can remove feasible approaches to another and the order of insertion becomes consequential. Despite having the same median inter-piece spacing of 37mm, TangramBench-Cluster instances have a median enclosure of only 9%, com-

pared with 57% for TangramBench-Puzzle instances. This increased enclosure makes TangramBench-Puzzle substantially more constrained, as feasible approach directions are reduced and successful assembly depends more strongly on coordinated sequencing of the pieces. All configurations are generated procedurally from fixed seeds, allowing disjoint training and evaluation pools of arbitrary size to be reproduced. Here, we provide approximately 1M, 1.1M, and 955k training configurations for TangramBench-Trio, Cluster, and Puzzle, respectively. Additionally, we provide 258 held-out configurations for each evaluation[1].

For evaluation, we provide a MuJoCo [30] implementation of TangramBench with mesh-based collision between pieces. Following the object-centric formulation of the benchmark, each simulated piece is directly actuated in its planar degrees of freedom. Velocity actuators realize commanded body-frame translational and angular velocities, allowing algorithms to focus on coordination between objects rather than on the kinematics of a particular robot platform. Finally, TangramBench provides a common set of evaluation metrics, including assembly success, position and yaw error, inter-piece contact rate, task completion time, and computation time. We also report the fraction of pieces within 5cm of their assigned targets. To support evaluation on physical hardware, we additionally provide a real-world counterpart using 3D-printed pieces with footprints matching the simulated models. Each piece contains a tapered insertion socket at its centroid that can be engaged by a custom-designed, square-shaped peg manipulation tool with a chamfered tip mounted to the robot (see in Fig. 2). This interface intentionally abstracts away grasp acquisition and constrains the interaction to planar translation and rotation, preserving the object-level planning problem studied in simulation while reducing the sim-to-real gap. The robot can therefore move one piece at a time using the same planar commands produced by the object-centric policy.

IV. Graph networks for Object-centric Multi-agent Assembly (GOMA)

Here, we provide an overview of GOMA, seen in Fig. 3.

A. Policy Architecture

We use separate policy and critic GNNs, following the edge, node, and global update structure of graph networks [28]. We denote the encoders by $\mathcal{E}_\theta^\pi$ and $\mathcal{E}_\phi^V$, with policy parameters θ and critic parameters ϕ. Within either encoder, learned embedding maps $\mathcal{B}_v$, $\mathcal{B}_e$, and $\mathcal{B}_g$ project raw node features x_i, edge features e_{ji}, and global features u to dimension H, which gives the initial node, edge, and global embeddings

$$\mathbf{h}_i^{(0)} = \mathcal{B}_v(x_i), \quad \mathbf{z}_{ji}^{(0)} = \mathcal{B}_e(e_{ji}), \quad \mathbf{g}^{(0)} = \mathcal{B}_g(u). \tag{2}$$

Embedding and update maps are residual multilayer perceptrons (MLPs).

[1]The camera-ready version will link the repository containing the generators, assets, and benchmark splits.

Each message-passing round updates edges from their previous embeddings, the sender and receiver embeddings, and the global context. Node updates combine the current node embedding with sums of incoming and outgoing edge embeddings, and the global update aggregates the updated nodes and edges. Writing $\mathcal{F}_e$, $\mathcal{F}_v$, and $\mathcal{F}_g$ for these maps and $[\cdot]$ for concatenation, a message-passing round k computes

$$\begin{aligned}
\mathbf{z}_{ji}^{(k+1)} &= \mathcal{F}_e\big([\mathbf{z}_{ji}^{(k)}, \mathbf{h}_j^{(k)}, \mathbf{h}_i^{(k)}, \mathbf{g}^{(k)}]\big), \\
\mathbf{m}_{i,\text{in}}^{(k+1)} &= \textstyle\sum_{j\to i} \mathbf{z}_{ji}^{(k+1)}, \qquad \mathbf{m}_{i,\text{out}}^{(k+1)} = \textstyle\sum_{i\to j} \mathbf{z}_{ij}^{(k+1)}, \\
\mathbf{h}_i^{(k+1)} &= \mathcal{F}_v\big([\mathbf{h}_i^{(k)}, \mathbf{m}_{i,\text{in}}^{(k+1)}, \mathbf{m}_{i,\text{out}}^{(k+1)}, \mathbf{g}^{(k)}]\big), \\
\mathbf{g}^{(k+1)} &= \mathcal{F}_g\Big(\Big[\mathbf{g}^{(k)}, \textstyle\sum_i \mathbf{h}_i^{(k+1)}, \sum_{j\to i} \mathbf{z}_{ji}^{(k+1)}\Big]\Big).
\end{aligned} \tag{3}$$

The global sums include all nodes and edges in $\mathcal{G}_t$, and each update map is shared across entities of the corresponding type.

Let $\mathbf{h}_i^\pi$ be the final piece embedding from $\mathcal{E}_\theta^\pi$ and $\mathbf{g}^V$ the final global embedding from $\mathcal{E}_\phi^V$. The policy head $\mathcal{H}_\theta^\pi$ is shared across pieces, while the critic head $\mathcal{H}_\phi^V$ outputs one scalar for the scene. With action scaling factors $a_{\max}$, the network outputs are

$$\mu_{\theta,i} = a_{\max} \odot \tanh\big(\mathcal{H}_\theta^\pi(\mathbf{h}_i^\pi)\big), \quad V_\phi(\mathcal{G}) = \mathcal{H}_\phi^V(\mathbf{g}^V). \tag{4}$$

Each piece uses a diagonal Gaussian action distribution with mean $\mu_{\theta,i}$ and a learned standard deviation vector σ_θ shared across pieces. Table II summarizes our hyperparameters.

B. Policy Training

We train on parallel simulation episodes with sampled target configurations and randomized initial piece poses. All pieces are actuated concurrently, with each command held over a fixed interval of simulation steps. The reward combines progress and pose accuracy for each piece with contact penalties, workspace penalties, and a shared incentive to complete the assembly. We define the distance error $d_i = \|p_i^\star - p_i\|$ and let ϵ_i be the wrapped orientation error. With that, we define I_i as the completion indicator that $d_i < \tau_p$ and $|\epsilon_i| < \tau_\vartheta$. We use $\tau_p = 0.01\,\text{m}$ and $\tau_\vartheta = 0.1\,\text{rad}$ as the completion thresholds. Based on this, the individual reward is defined as

$$\begin{aligned}
r_i^{\text{ind}} = w_v \frac{(p_i^\star - p_i)^\top v_i}{d_i + \varepsilon} \frac{\|v_i\|}{v_{\text{ref}}} - w_\vartheta(1 - \cos\epsilon_i) \\
+ w_s I_i - w_c \textstyle\sum_{j\neq i} c_{ij} - w_b b_i.
\end{aligned} \tag{5}$$

The progress term projects velocity onto the target direction and scales it by speed relative to v_{ref}. The contact indicator c_{ij} is accumulated over simulation steps within each control interval, capturing transient contacts. Contacts do not terminate an episode, allowing the policy to correct configurations after an interaction. The contact cost discourages interference between moving pieces, while the pose reward favors closing the gaps required for compact assemblies. The workspace term b_i combines positive-part penalties for crossing the rectangular workspace limits and entering a margin near its boundary. The sparse pose reward is supplied at every step that the piece remains within both target tolerances.

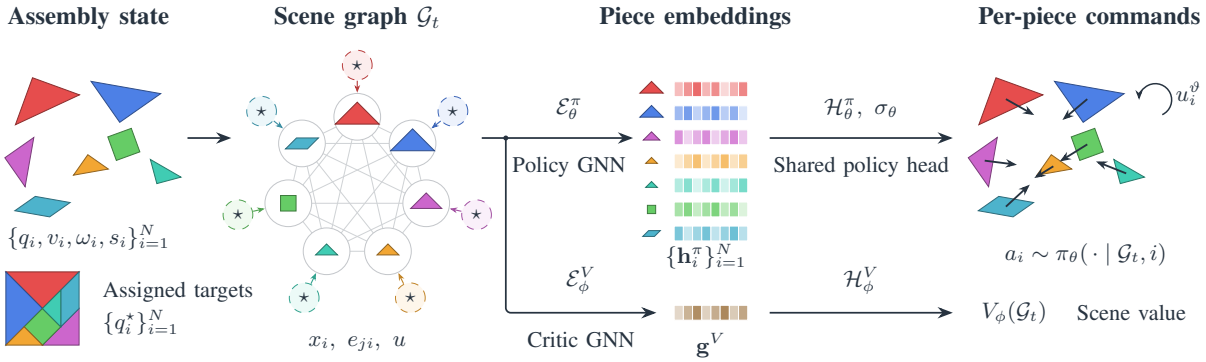


Fig. 3: GOMA maps a graph of pieces and assigned target poses to per-piece velocity commands. Independent graph encoders produce piece embeddings for a shared policy head and a global embedding for the critic.

The shared reward penalizes the maximum positional error and rewards completion of all target poses,

$$r_i = r_i^{\text{ind}} - w_d \max_j d_j + w_T \prod_{j=1}^N I_j. \tag{6}$$

These shared terms keep the reward sensitive to unfinished placements even when a piece has reached its own target.

We optimize the policy with the per-agent clipped objective of following MAPPO [14] with the reward of Eq. (6). Generalized advantage estimation (GAE) computes a separate advantage for each piece from its reward $r_{i,t}$ and the shared scene baseline $V_\phi(\mathcal{G}_t)$. Each likelihood ratio covers the three command components of one piece, and the clipped surrogate is averaged over pieces and scene transitions. This separates the clipping across pieces, whereas joint-action PPO would couple them through a single ratio for all piece commands. The critic predicts one scene value and is trained toward the mean of the per-piece return targets. This keeps individual rewards in the policy updates while sharing the baseline across pieces.

V. Experiments

In this section, we evaluate GOMA on TangramBench to study how multiple puzzle pieces can be moved toward their targets in a coordinated manner. In particular, we investigate whether an object-centric multi-agent formulation provides useful structure for this problem and which components are responsible for its performance. Our experiments address three questions: (i) *what architectural structure is needed to learn coordinated multi-object assembly?* (ii) *how do the individual reward terms affect the learned coordination behaviour?* and (iii) *how does the resulting policy compare with classical motion-planning approaches on increasingly constrained assembly tasks?*

A. Experimental Setup

We primarily run our experiments and evaluations in simulation. Here we use the MuJoCo implementation of TangramBench, trained on the GPU with 1024 instances in parallel. The physics step is 10ms and the policy acts at 10Hz. Contacts between pieces are recorded at every physics step. Every episode draws a new target configuration and start poses from the training set. Evaluation uses the held-out configurations.

B. What Structure Enables Coordinated Assembly?

First, to understand the effect of each element in our architecture, we isolate the main components of GOMA and analyze training and evaluation performance on TangramBench-Trio. We compare GOMA against three architectural variants. Single-agent PPO discards both the graph and the multi-agent formulation. Here, an MLP maps the concatenated state of all pieces and targets, in a fixed order, to the velocities of all pieces, and is trained using a single clipped objective and the mean per-piece reward. MLP-MAPPO retains the per-agent formulation of Sec. IV-B, including a shared policy, per-piece advantages, and a centralized critic, but replaces the graph representation with an MLP operating on each piece's local state. Finally, GOMA-J retains the graph actor and critic, but replaces the per-agent MAPPO objective with a single joint team reward. Fig. 4-top shows the evolution of task success, the fraction of pieces within 5cm of their targets, position and yaw errors, and contact rate throughout training for all four architectures, averaged over five seeds. Here, we can see a clear progression as structure is introduced into the policy. Single-agent PPO, which uses neither the graph representation nor the multi-agent decomposition, fails to achieve meaningful task success and retains large position and yaw errors throughout training. Introducing the multi-agent formulation alone already provides a substantial improvement. MLP-MAPPO brings almost 90% of pieces within 5cm of their targets, although its overall success plateaus at approximately 50%. Adding relational message passing closes this gap, with both graph-based variants, GOMA and GOMA-J, converging to approximately 90% success and substantially lower final position errors. All methods maintain low contact rates, indicating that the differences arise primarily from their ability to coordinate precise placement rather than simply avoid collisions. Interestingly, GOMA and GOMA-J perform very similarly across all metrics, suggesting that while the per-piece advantages of MAPPO gives the policy a useful inductive bias, explicitly modeling relations between pieces through the GNN is essential for success.

C. What Drives Coordinated Behavior?

Next, we study how the reward formulation influences the behavior learned by GOMA. We consider two ablations here and evaluate them on TangramBench-Clusters. In "GOMA, no team reward", we remove the shared terms of Eq. (6), i.e., the maximum-distance penalty and the global completion bonus, leaving each agent to optimize only its individual reward. In "GOMA, no contact cost", we remove the inter-piece contact penalty from Eq. (5). All other reward terms, architectures, and training parameters remain unchanged. In Fig. 4-bottom we again show the evolution of task success, the fraction of pieces within 5cm of their targets, position and yaw errors, and contact rate throughout training, all averaged over five seeds. All three variants learn to move most pieces close to their targets, reaching a similar fraction of pieces within 5cm and comparable final pose errors. The resulting behaviors, however, are considerably different. Removing the contact cost leads to frequent inter-piece contact throughout training, whereas both variants trained with this penalty reduce contacts to near zero. Removing the shared reward has little effect on collision avoidance or coarse goal reaching, but results in lower final assembly success. This suggests that the individual rewards are sufficient to drive pieces toward their respective targets, while the shared terms help translate this individual progress into completion of the assembly as a whole.

We further evaluate these reward variants as the number of pieces increases. Fig. 5 compares success on assemblies containing three, five, six, and seven pieces, testing whether the contribution of the shared reward becomes more pronounced as the number of simultaneously interacting pieces grows. We trained on a scene with seven objects and used this trained policy for evaluation of all different piece counts.

D. How Does Learned Coordination Compare with Classical Planning?

Finally, we compare GOMA against myopic and sampling-based motion planning. An attractive-only artifical potential field (APF) [31] serves as the myopic baseline. It drives every piece independently and straight toward its target, using the same tracking controller as the planners. *Joint rapidly-exploring random tree (RRT)* (RRT-J) plans one path for all pieces at once with RRT-Connect [21] in the 21-dimensional configuration space and with mesh collision checks. *Per-piece RRT* (RRT-P) plans a 3-dimensional path for every piece against the start poses of the others and executes all paths at once. Both planners are also evaluated with a replan mode, where a new plan is generated every 5s. This is added so that they can react to the movements of other pieces. Under one-piece-at-a-time execution, the joint plan is followed piece by piece, and the per-piece paths are planned in sequence with the earlier pieces assumed at their goals. All methods receive the same target assignments and the same initial configurations. The planners are given the 117 of 258 held-out instances whose start arrangement passes their collision check and a 150 s episode, so that planning time does not decide the outcome. In Tab. I we compare the methods on TangramBench-Cluster and TangramBench-Puzzle. We report assembly success, final position error, the fraction of control steps with inter-piece contact, the median time to completion, and the computation time at inference.

TABLE I: GOMA against the APF controller and joint (RRT-J) and per-piece (RRT-P) RRT planning, planned once or replanned every 5 s, on the held-out TangramBench-Cluster and TangramBench-Puzzle.

Method	Succ. (%)	Err. (cm)	Contact (%)	Time (s)	Compute
TangramBench-Cluster					
GOMA (ours)	75	1.0	4	8	0.68 ms
APF	3	3.8	99	80	≈ 0
RRT-J	73	1.9	44	20	774 ms
RRT-J +replan	98	0.3	12	21	49 ms
RRT-P	3	4.0	98	119	42 ms
RRT-P +replan	9	3.2	95	85	6.7 ms
TangramBench-Puzzle					
GOMA (ours)	41	1.7	4	24	0.68 ms
APF	0	4.3	100	–	≈ 0
RRT-J	58	3.3	59	24	6.8 s
RRT-J +replan	88	1.5	37	43	1.1 s
RRT-P	0	4.5	99	–	48 ms
RRT-P +replan	0	4.1	99	–	0.20 ms

These results highlight the importance of coordination. Crucially, APF and RRT-P assemble at most 9% of the clusters and none of the puzzles, with inter-piece contact during nearly every control step. Joint planning with replanning achieves the highest success under concurrent execution (98% on TangramBench-Cluster and 88% on TangramBench-Puzzle), compared with 75% and 41% for GOMA. However, its contact rates are also higher. 12% and 37%, respectively, compared with 4% for GOMA on both benchmarks. Additionally, GOMA requires approximately 0.68ms per inference and achieves median completion times of 8 s on TangramBench-Cluster and 24 s on TangramBench-Puzzle, compared with 21 s and 43s for RRT-J with replanning. Thus, GOMA offers fast, low-contact execution, while joint planning with replanning remains more reliable at completing tightly constrained assemblies.

E. Zero-shot Sim2real transfer

To demonstrate the applicability of GOMA beyond simulation, we perform a qualitative zero-shot sim-to-real demonstration on a Franka FR3 manipulator, without additional policy training or fine-tuning. The robot is controlled using the CRISP Cartesian impedance controller and is equipped with the custom square-peg manipulation tool introduced in Section III. Piece poses are estimated using a hand–eye calibrated camera setup consisting of a scene-mounted Intel RealSense D435i RGB-D camera for global pose estimation and a wrist-mounted RealSense D405 for close-range localization of the insertion socket. We estimate the pose of each piece using FoundationPose [32], with HSV color segmentation providing an individual mask for each of the seven pieces. Known geometric offsets transform the estimated model poses to the corresponding piece centroids,

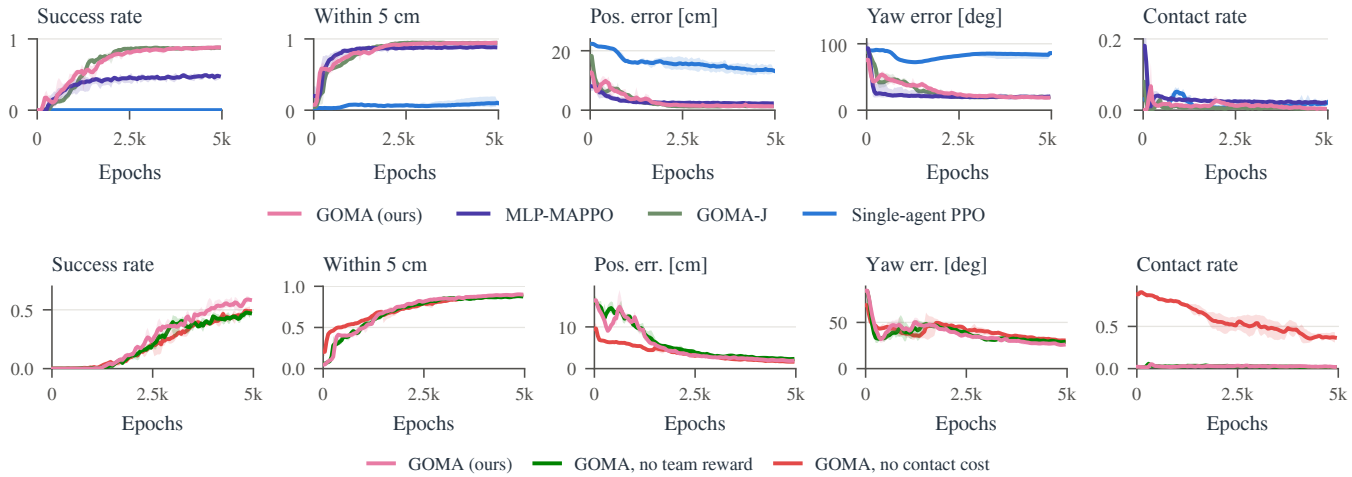


Fig. 4: Policy ablations. On the top row, we show the architecture ablation on TangramBench-Trio. Here we report task success, fraction of pieces within 5cm of their targets, position and yaw errors, and contact rate throughout training. On the bottom, the effect of the reward formulation. We ablate the shared team reward and inter-piece contact cost.

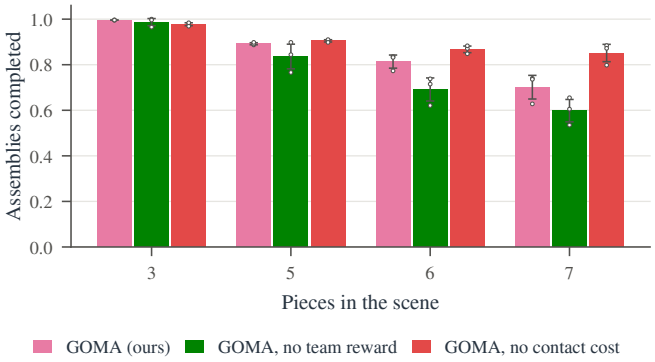


Fig. 5: Success comparison on assemblies containing a different piece count.

while a geometric consistency filter rejects estimates that would imply physically impossible overlaps.

Since the physical robot can manipulate only one piece at a time, we execute the multi-agent policy sequentially. A round-robin strategy selects among unfinished pieces, allowing a piece to remain temporarily in place and be revisited in subsequent rounds. For each selected piece, the wrist-mounted camera first localizes its insertion socket, after which the robot engages the tool and executes the motion predicted by the corresponding agent. During execution, the pose of the manipulated piece is updated from the robot's forward kinematics, while the poses of the remaining pieces are obtained from perception. The resulting object poses are used to reconstruct the interaction graph before each policy query. Fig. 1 shows the resulting real-world assembly sequence, illustrating that the policy can be deployed on the physical setup without additional real-world training.

VI. Conclusion

In this work, we studied planar geometric assembly as a coordinated multi-object manipulation problem. We introduced TangramBench, a procedural Tangram benchmark with graded levels of geometric coupling, together with simulation and a corresponding real-robot setup. On this benchmark, we formulated assembly from an object-centric multi-agent perspective and proposed GOMA, where each piece is represented as an agent and relational information is exchanged through a graph neural network. Our experiments investigate which components of this formulation enable coordinated behavior and compare the resulting policy with classical motion-planning approaches across increasingly constrained assembly configurations.

Our formulation intentionally isolates coordination from several complementary challenges in robotic assembly. In particular, the target pose and piece-to-target assignments are provided. We therefore do not address the problem of inferring a Tangram decomposition from a target silhouette and determining which piece belongs to each region. Incorporating these capabilities would connect the proposed benchmark more directly to the complete Tangram-solving problem and represents an interesting direction for future work. Similarly, while our simulated formulation permits all pieces to move concurrently, the current physical setup uses a single robot and therefore executes the multi-agent policy sequentially. Extending the benchmark to bimanual or multi-robot systems would allow the concurrent coordination learned by the policy to be evaluated directly in the physical world. More broadly, TangramBench abstracts away traditional nonprehensile pushing and constrains manipulation to planar motion through the insertion tool, allowing us to focus specifically on the interactions between pieces. Future work could relax these assumptions by incorporating pushing, richer contact dynamics, partial observability, and uncertainty in perception, as well as extending the procedural generation framework beyond Tangram to more general planar assemblies and ultimately full 3D assembly problems.

References

[1] R. A. Knepper, T. Layton, J. Romanishin, and D. Rus, "Ikeabot: An autonomous multi-robot coordinated furniture assembly system," in *2013 IEEE Int. Conf. on robotics and automation*. IEEE, 2013, pp. 855–862.

[2] D. Kirschner, R. Velik, S. Yahyanejad, M. Brandstötter, and M. Hofbaur, "Yumi, come and play with me! a collaborative robot for piecing together a tangram puzzle," in *Int. Conf. on Interactive Collaborative Robotics*. Springer, 2016, pp. 243–251.

[3] C. R. Garrett, R. Chitnis, R. M. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," *Annu. Rev. Control. Robotics Auton. Syst.*, vol. 4, pp. 265–293, 2021.

[4] Y. Tian, K. D. D. Willis, B. Al Omari, J. Luo, P. Ma, Y. Li, F. Javid, E. Gu, J. Jacob, S. Sueda, H. Li, S. Chitta, and W. Matusik, "ASAP: Automated sequence planning for complex robotic assembly with physical feasibility," in *2024 IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024, pp. 4380–4386.

[5] J. E. King, M. Cognetti, and S. S. Srinivasa, "Rearrangement planning using object-centric and robot-centric action spaces," in *2016 IEEE Int. Conf. on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 3940–3947.

[6] K. Ren, G. Wang, A. S. Morgan, L. E. Kavraki, and K. Hang, "Object-centric kinodynamic planning for nonprehensile robot rearrangement manipulation," *IEEE Transactions on Robotics*, 2025.

[7] F. Bai, F. Meng, J. Liu, J. Wang, and M. Q.-H. Meng, "Hierarchical policy with deep-reinforcement learning for nonprehensile multiobject rearrangement," *Biomimetic Intelligence and Robotics*, vol. 2, no. 3, p. 100047, 2022.

[8] N. Funk, G. Chalvatzaki, B. Belousov, and J. Peters, "Learn2Assemble with structured representations and search for robotic architectural construction," in *Proceedings of the 5th Conf. on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 1401–1411.

[9] S. K. S. Ghasemipour, D. Freeman, B. David, S. S. Gu, S. Kataoka, and I. Mordatch, "Blocks assemble! Learning to assemble with large-scale structured reinforcement learning," in *Proceedings of the 39th Int. Conf. on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 7435–7469.

[10] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, R. Barták, and E. Boyarski, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proceedings of the Twelfth Int. Symposium on Combinatorial Search, SOCS 2019, Napa, California, 16-17 July 2019*. AAAI Press, 2019, pp. 151–158.

[11] Q. Li, F. Gama, A. Ribeiro, and A. Prorok, "Graph neural networks for decentralized multi-robot path planning," in *2020 IEEE/RSJ Int. Conf. on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 11 785–11 792.

[12] S. Nayak, K. Choi, W. Ding, S. Dolan, K. Gopalakrishnan, and H. Balakrishnan, "Scalable multi-agent reinforcement learning through intelligent information aggregation," in *Proceedings of the 40th Int. Conf. on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 25 817–25 833.

[13] M. Lai, K. Go, Z. Li, T. Kröger, S. Schaal, K. Allen, and J. Scholz, "RoboBallet: Planning for multirobot reaching with graph neural networks and reinforcement learning," *Sci. Robotics*, vol. 10, no. 106, p. eads1204, 2025.

[14] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, vol. 35. Curran Associates, Inc., 2022, pp. 24 611–24 624.

[15] C. Zhao, C. Jiang, L. Luo, G. Zhang, H. Yu, M. Y. Wang, and Q. Chen, "Master rules from chaos: Learning to reason, plan, and interact from chaos for tangram assembly," in *2025 IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2025, pp. 11 503–11 509.

[16] Y. Zong and C. Tan, "Tangramsr: Can vision-language models reason in continuous geometric space?" *arXiv preprint arXiv:2602.05570*, 2026.

[17] D. Liu, J. Kuang, Y. Li, Y. Li, D. Yin, H. Cao, X. Sun, Y. Shen, H.-T. Zheng, L. Lin *et al.*, "Tangrampuzzle: Evaluating multimodal large language models with compositional spatial reasoning," *arXiv preprint arXiv:2601.16520*, 2026.

[18] Y. Tian, J. Xu, Y. Li, J. Luo, S. Sueda, H. Li, K. D. D. Willis, and W. Matusik, "Assemble them all: Physics-based planning for generalizable assembly by disassembly," *ACM Trans. Graph.*, vol. 41, no. 6, pp. 278:1–278:15, 2022.

[19] Y. Tian, J. Jacob, Y. Huang, J. Zhao, E. L. Gu, P. Ma, A. Zhang, F. Javid, B. Romero, S. Chitta, S. Sueda, H. Li, and W. Matusik, "Fabrica: Dual-arm assembly of general multi-part objects via integrated planning and learning," in *9th Annual Conf. on Robot Learning*, 2025.

[20] V. N. Hartmann, A. Orthey, D. Driess, O. S. Oguz, and M. Toussaint, "Long-horizon multi-robot rearrangement planning for construction assembly," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 239–252, 2023.

[21] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium Conf.. IEEE Int. Conf. on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, pp. 995–1001.

[22] J. A. Haustein, I. Arnekvist, J. Stork, K. Hang, and D. Kragic, "Learning manipulation states and actions for efficient non-prehensile rearrangement planning," arXiv preprint arXiv:1901.03557, 2019.

[23] K. Hansel, J. Urain, J. Peters, and G. Chalvatzaki, "Hierarchical policy blending as inference for reactive robot control," in *2023 IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2023, pp. 10 181–10 188.

[24] C. Shu, A. Kim, and S. Park, "Subassembly to full assembly: Effective assembly sequence planning through graph-based reinforcement learning," in *2025 IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2025, pp. 14 946–14 952.

[25] Z. Wang, W. Liu, J. Wang, G. Vallat, F. Shi, S. Parascho, and M. Kamgarpour, "Learning to assemble with alternative plans," *ACM Trans. Graph.*, vol. 44, no. 4, pp. 101:1–101:16, 2025.

[26] J. Huang, G. Zhan, Q. Fan, K. Mo, L. Shao, B. Chen, L. Guibas, and H. Dong, "Generative 3D part assembly via dynamic graph learning," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 6315–6326.

[27] S. Kataoka, Y. Chung, S. K. S. Ghasemipour, P. Sanketi, S. S. Gu, and I. Mordatch, "Bi-manual block assembly via sim-to-real reinforcement learning," arXiv preprint arXiv:2303.14870, 2023.

[28] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, "Relational inductive biases, deep learning, and graph networks," arXiv preprint arXiv:1806.01261, 2018.

[29] F. M. Yamada, H. C. Batagelo, J. P. Gois, and H. Takahashi, "Tangan: solving tangram puzzles using generative adversarial network: Fm yamada et al." *Applied Intelligence*, vol. 55, no. 7, p. 511, 2025.

[30] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control." in *IROS*. IEEE, 2012, pp. 5026–5033.

[31] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The international journal of robotics research*, vol. 5, no. 1, pp. 90–98, 1986.

[32] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "Foundationpose: Unified 6d pose estimation and tracking of novel objects," in *2024 IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 17 868–17 879.

APPENDIX
ALGORITHM HYPERPARAMETERS

TABLE II: Hyperparameters for GOMA.

Algorithm setting	**Value**
Latent and hidden dimension H	256
Message-passing rounds K	1
Node embedding and update blocks	9
Edge & global embedding & update blocks	8
Policy and critic head blocks	1
Policy learning rate	$2\times10^{-5} \rightarrow 10^{-6}$
Critic learning rate	$6\times10^{-5} \rightarrow 3\times10^{-6}$
Discount factor γ	0.99
Advantage parameter λ	0.9
Policy and value clipping η	0.1
Update epochs	3
Minibatch size	4096
Entropy coefficient	0.0
Maximum gradient norm	5
Rollout length	100
Parallel environments	1024